

SSPA-1: Complete Akamai Deployment Automation with ServiceNow Integration

What This Whole System Does (Plain English)

Application teams want to push new Akamai CDN config versions to production WITHOUT needing an Akamai engineer on the call every time. This automation:

1. **Checks ServiceNow** — Is the Change Request approved, in Implement state, and within its scheduled window? If not, stop immediately.
 2. **Compares Akamai versions** — What's currently live vs what we're deploying? Shows the diff for review.
 3. **Activates on Akamai** — Pushes the new config version via the PAPI API with proper EdgeGrid auth.
 4. **Monitors until done** — Polls every 10 seconds for up to 60 minutes until activation completes.
 5. **Writes results back to ServiceNow** — Updates the ctask with version, timestamp, pass/fail, error logs if any.
 6. **Rolls back automatically** — If anything fails, activates the previous stable version.
-

Complete Project Folder Structure

```
sspa_akamai_automation/
├─ ansible.cfg
├─ requirements.txt           ← Python deps: edgegrid-python,
requests, pysnow
├─ collections/
│   └─ requirements.yml      ← Ansible collections: servicenow.itsm
├─ inventories/
│   └─ staging/
│       ├── hosts.yml
│       └─ group_vars/all/
│           ├── vars.yml     ← non-secret variables
│           └─ vault.yml     ← encrypted secrets (ansible-vault)
│   └─ production/
│       ├── hosts.yml
│       └─ group_vars/all/
│           ├── vars.yml
│           └─ vault.yml
├─ playbooks/
│   └─ deploy_akamai_config.yml ← the main playbook
├─ roles/
│   └─ snow_chg_validate/    ← SSPA-4 + SSPA-8: read-only CHG gate
check
│   │   └─ tasks/
│   │       └─ main.yml
│   └─ snow_ctask_update/    ← SSPA-14: write deployment results
back to ctask
│   │   └─ tasks/
│   │       └─ main.yml
│   └─ akamai_property/     ← Core Akamai PAPI automation
│       ├── defaults/
│       │   └─ main.yml
│       ├── tasks/
│       │   ├── main.yml
│       │   ├── preflight.yml ← concurrency check
│       │   ├── create_version.yml ← clone active version
│       │   ├── diff.yml      ← generate rule tree diff
│       │   ├── activate.yml  ← push to staging/production
│       │   ├── poll_status.yml ← wait for ACTIVE/FAILED/ABORTED
│       │   └─ rollback.yml   ← activate previous stable version
│       ├── templates/
│       │   └─ edgerc.j2      ← renders .edgerc from vault at
runtime
│   └─ files/
```

```
| └─ akamai_papi.py      ← EdgeGrid signing wrapper (Python)
└─ artifacts/           ← diff outputs saved here, gitignored
```

File 1: ansible.cfg

```
ini

[defaults]
inventory          = inventories/staging
roles_path         = roles
collections_paths  = collections
vault_password_file = .vault_pass
stdout_callback    = yaml
host_key_checking  = False

[privilege_escalation]
become = False
```

File 2: requirements.txt

```
edgegrid-python>=1.3.1
requests>=2.28.0
pysnow>=0.7.2
```

File 3: collections/requirements.yml

```
yaml

---
collections:
  - name: servicenow.itsm
    version: ">=2.0.0"
  - name: servicenow.servicenow
    version: ">=1.0.17"
  - name: ansible.builtin
```

File 4: inventories/staging/hosts.yml

```
yaml
```

```
---
all:
  hosts:
    localhost:
      ansible_connection: local
      ansible_python_interpreter: "{{ ansible_playbook_python }}"
```

File 5: inventories/staging/group_vars/all/vars.yml

```
yaml

---
# Environment identifier - used as physical guardrail
akamai_network: "STAGING"      # STAGING or PRODUCTION
environment_name: "staging"

# Akamai Property details (get from Akamai Control Center)
akamai_property_id: "prp_XXXXXX"      # e.g. prp_123456
akamai_contract_id: "ctr_XXXXXX"      # e.g. ctr_1-ABC123
akamai_group_id: "grp_XXXXXX"         # e.g. grp_98765

# Akamai API host (from your .edgerc / API client)
akamai_host: "akab-xxxx-xxxx.luna.akamaiapis.net"

# ServiceNow instance
snow_instance: "pncint.service-now.com"

# Polling config (matches the Confluence doc spec)
activation_poll_interval: 10          # seconds between polls
activation_poll_max_attempts: 360     # 360 * 10s = 60 minutes

# Rollback config
rollback_on_failure: true
use_fast_fallback: true

# Paths
edgerc_path: "/tmp/.edgerc_akamai_{{ ansible_date_time.epoch }}"
artifacts_dir: "{{ playbook_dir }}/../artifacts"
```

File 6: inventories/production/group_vars/all/vars.yml

```
yaml
```

```

---
# This is the ONLY file that differs significantly from staging
akamai_network: "PRODUCTION"
environment_name: "production"

akamai_property_id: "prp_XXXXXX"    # same property
akamai_contract_id: "ctr_XXXXXX"
akamai_group_id: "grp_XXXXXX"
akamai_host: "akab-yyyy-yyyy.luna.akamaiapis.net"    # DIFFERENT API client for

snow_instance: "pncint.service-now.com"

activation_poll_interval: 10
activation_poll_max_attempts: 360

rollback_on_failure: true
use_fast_fallback: true

edgerc_path: "/tmp/.edgerc_akamai_{{ ansible_date_time.epoch }}"
artifacts_dir: "{{ playbook_dir }}/../artifacts"

```

File 7: inventories/staging/group_vars/all/vault.yml

(Encrypted with ansible-vault - shown decrypted here for reference)

```

yaml
---
# Akamai EdgeGrid credentials (4 values from IAM API client)
vault_akamai_client_token: "akab-xxxxxxxxxxxxxxxxxxxxxx"
vault_akamai_access_token: "akab-xxxxxxxxxxxxxxxxxxxxxx"
vault_akamai_client_secret: "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx="
# Note: host comes from vars.yml (not sensitive)

# ServiceNow credentials
vault_snow_username: "svc_ansible_automation"
vault_snow_password: "xxxxxxxxxxxxxxxxxxxxxx"
# OR for OAuth:
vault_snow_client_id: "xxxxxxxxxxxxxxxxxxxxxx"
vault_snow_client_secret: "xxxxxxxxxxxxxxxxxxxxxx"

```

File 8: roles/akamai_property/templates/edgerc.j2

```

ini

```

```
[default]
client_secret = {{ vault_akamai_client_secret }}
host = {{ akamai_host }}
access_token = {{ vault_akamai_access_token }}
client_token = {{ vault_akamai_client_token }}
```

File 9: roles/akamai_property/files/akamai_papi.py

(The Python EdgeGrid signing wrapper — the core piece Ansible uri module cannot do)

```
python
```



```
"""
```

```
Get the currently active version on STAGING and PRODUCTION.  
Returns dict with stagingVersion and productionVersion.
```

```
Note: /versions/latest returns a 302 redirect - we read from  
the property object directly instead.
```

```
"""
```

```
url = (f"{base_url}/papi/v1/properties/{property_id}"  
       f"?contractId={contract_id}&groupId={group_id}")  
resp = session.get(url)  
resp.raise_for_status()  
data = resp.json()  
prop = data["properties"]["items"][0]  
return {  
    "stagingVersion": prop.get("stagingVersion"),  
    "productionVersion": prop.get("productionVersion"),  
    "latestVersion": prop.get("latestVersion"),  
}
```

```
def create_version(session, base_url, property_id,  
                  contract_id, group_id, base_version: int) -> int:
```

```
"""
```

```
Clone base_version into a new editable version.  
Returns the new version number.
```

```
"""
```

```
url = make_url(base_url, property_id, contract_id, group_id, "/versions")  
payload = {"createFromVersion": base_version}  
# contractId/groupId go in the query string, NOT the body  
resp = session.post(url, json=payload,  
                    headers={"Content-Type": "application/json"})  
resp.raise_for_status()  
# Response contains a link like /papi/v1/.../versions/5  
link = resp.json()["versionLink"]  
new_version = int(link.rstrip("/").split("/")[-1])  
return new_version
```

```
def get_rules(session, base_url, property_id,  
              contract_id, group_id, version: int) -> dict:
```

```
"""
```

```
Fetch the full rule tree JSON for a specific version.  
Uses versioned Accept header to pin schema format.
```

```
"""
```

```
url = make_url(base_url, property_id, contract_id, group_id,  
               f"/versions/{version}/rules")  
resp = session.get(url, headers={
```

```

        "Accept": "application/vnd.akamai.papirules.latest+json"
    })
    resp.raise_for_status()
    return resp.json()

def activate(session, base_url, property_id,
             contract_id, group_id, version: int,
             network: str, email: str,
             note: str = "Automated deployment via Ansible SSPA",
             fast_fallback: bool = True) -> str:
    """
    Activate version to STAGING or PRODUCTION.
    Returns the activation ID for polling.
    network must be 'STAGING' or 'PRODUCTION'
    """
    url = make_url(base_url, property_id, contract_id, group_id,
                  "/activations")
    payload = {
        "propertyVersion": version,
        "network": network,
        "note": note,
        "notifyEmails": [email],
        "acknowledgeAllWarnings": True,    # soft warnings auto-acknowledged
        "useFastFallback": fast_fallback,  # enable 60-min fast rollback window
    }
    resp = session.post(url, json=payload,
                       headers={"Content-Type": "application/json"})
    # 409 = another activation is already in progress (concurrency conflict)
    if resp.status_code == 409:
        raise RuntimeError(
            "CONCURRENCY_CONFLICT: Another activation is already "
            "in progress for this property. Wait for it to complete."
        )
    resp.raise_for_status()
    link = resp.json()["activationLink"]
    activation_id = link.rstrip("/").split("/")[-1]
    return activation_id

def get_activation_status(session, base_url, property_id,
                         contract_id, group_id,
                         activation_id: str) -> dict:
    """
    Poll the activation status.
    Returns dict with status: PENDING | ACTIVE | FAILED | ABORTED
    """

```

```

url = (f"{base_url}/papi/v1/properties/{property_id}"
      f"/activations/{activation_id}"
      f"?contractId={contract_id}&groupId={group_id}")
resp = session.get(url)
resp.raise_for_status()
activations = resp.json()["activations"]["items"]
if not activations:
    raise RuntimeError(f"No activation found with id {activation_id}")
return activations[0]

def list_activations(session, base_url, property_id,
                     contract_id, group_id) -> list:
    """
    List all activations. Used for concurrency preflight check.
    If any activation is in PENDING status, we must not start another.
    """
    url = make_url(base_url, property_id, contract_id, group_id,
                  "/activations")
    resp = session.get(url)
    resp.raise_for_status()
    return resp.json()["activations"]["items"]

def main():
    parser = argparse.ArgumentParser(description="Akamai PAPI EdgeGrid wrapper")
    parser.add_argument("--operation", required=True,
                        choices=["get_active_version", "create_version",
                                "get_rules", "activate",
                                "get_activation", "list_activations"])
    parser.add_argument("--edgerc", required=True,
                        help="Path to .edgerc credential file")
    parser.add_argument("--section", default="default",
                        help="Section name in .edgerc")
    parser.add_argument("--property-id", required=True)
    parser.add_argument("--contract-id", required=True)
    parser.add_argument("--group-id", required=True)
    parser.add_argument("--version", type=int,
                        help="Property version number")
    parser.add_argument("--network",
                        choices=["STAGING", "PRODUCTION"],
                        help="Target network for activation")
    parser.add_argument("--activation-id",
                        help="Activation ID for status polling")
    parser.add_argument("--email", default="ansible-automation@example.com",
                        help="Notification email for activations")
    parser.add_argument("--note", default="Automated deployment via Ansible",

```

```
        help="Activation note/comment")
parser.add_argument("--fast-fallback", action="store_true", default=True,
                    help="Enable fast fallback (60-min rollback window)")
args = parser.parse_args()

session, base_url = get_session(args.edgerc, args.section)

try:
    if args.operation == "get_active_version":
        result = get_active_version(session, base_url,
                                    args.property_id,
                                    args.contract_id,
                                    args.group_id)

    elif args.operation == "create_version":
        if not args.version:
            print(json.dumps({"error": "--version required"}),
                  file=sys.stderr)
            sys.exit(1)
        new_ver = create_version(session, base_url,
                                args.property_id,
                                args.contract_id,
                                args.group_id,
                                args.version)
        result = {"new_version": new_ver}

    elif args.operation == "get_rules":
        if not args.version:
            print(json.dumps({"error": "--version required"}),
                  file=sys.stderr)
            sys.exit(1)
        result = get_rules(session, base_url,
                            args.property_id,
                            args.contract_id,
                            args.group_id,
                            args.version)

    elif args.operation == "activate":
        if not args.version or not args.network:
            print(json.dumps(
                {"error": "--version and --network required"}),
                  file=sys.stderr)
            sys.exit(1)
        act_id = activate(session, base_url,
                           args.property_id,
                           args.contract_id,
                           args.group_id,
```

```

        args.version,
        args.network,
        args.email,
        args.note,
        args.fast_fallback)
    result = {"activation_id": act_id}

elif args.operation == "get_activation":
    if not args.activation_id:
        print(json.dumps({"error": "--activation-id required"}),
              file=sys.stderr)
        sys.exit(1)
    result = get_activation_status(session, base_url,
                                   args.property_id,
                                   args.contract_id,
                                   args.group_id,
                                   args.activation_id)

elif args.operation == "list_activations":
    result = list_activations(session, base_url,
                              args.property_id,
                              args.contract_id,
                              args.group_id)

print(json.dumps(result))

except RuntimeError as e:
    print(json.dumps({"error": str(e)}), file=sys.stderr)
    sys.exit(2)
except requests.HTTPError as e:
    print(json.dumps({
        "error": str(e),
        "status_code": e.response.status_code,
        "body": e.response.text
    }), file=sys.stderr)
    sys.exit(3)

if __name__ == "__main__":
    main()

```

File 10: roles/akamai_property/defaults/main.yml

yaml

```
---
# Default values - override in vars.yml or at runtime
akamai_network: "STAGING"
activation_poll_interval: 10
activation_poll_max_attempts: 360
rollback_on_failure: true
use_fast_fallback: true
akamai_notify_email: "ansible-automation@pnc.com"
akamai_edgerc_section: "default"
artifacts_dir: "{{ playbook_dir }}/../artifacts"
```

File 11: roles/akamai_property/tasks/main.yml

(Orchestrates all task files in order)

```
yaml
---
- name: "Include preflight checks"
  ansible.builtin.include_tasks: preflight.yml

- name: "Include create version"
  ansible.builtin.include_tasks: create_version.yml

- name: "Include diff generation"
  ansible.builtin.include_tasks: diff.yml

- name: "Include activation"
  ansible.builtin.include_tasks: activate.yml

- name: "Include poll status"
  ansible.builtin.include_tasks: poll_status.yml
```

File 12: roles/akamai_property/tasks/preflight.yml

(Concurrency check + credential setup)

```
yaml
```

```
---
# — Step 0a: Render .edgerc from vault, delete after play —
- name: "Create artifacts directory"
  ansible.builtin.file:
    path: "{{ artifacts_dir }}"
    state: directory
    mode: "0755"

- name: "Render temporary .edgerc from vault"
  ansible.builtin.template:
    src: edgerc.j2
    dest: "{{ edgerc_path }}"
    mode: "0600"
  no_log: true    # CRITICAL: never log credential file contents

# — Step 0b: Concurrency preflight —
- name: "List current activations (concurrency check)"
  ansible.builtin.command:
    cmd: >
      python3 {{ role_path }}/files/akamai_papi.py
      --operation list_activations
      --edgerc {{ edgerc_path }}
      --section {{ akamai_edgerc_section }}
      --property-id {{ akamai_property_id }}
      --contract-id {{ akamai_contract_id }}
      --group-id {{ akamai_group_id }}
  register: preflight_result
  changed_when: false
  no_log: true

- name: "Parse activation list"
  ansible.builtin.set_fact:
    current_activations: "{{ preflight_result.stdout | from_json }}"

- name: "Fail if activation already in progress"
  ansible.builtin.fail:
    msg: >-
      PREFLIGHT FAILED: Another activation is already PENDING for
      property {{ akamai_property_id }} on {{ akamai_network }}.
      Wait for it to complete before starting a new one.
  when: >
    current_activations | selectattr('status', 'equalto', 'PENDING')
    | selectattr('network', 'equalto', akamai_network)
    | list | length > 0

# — Step 0c: Get current active version —
```

```
- name: "Get currently active version info"
  ansible.builtin.command:
    cmd: >
      python3 {{ role_path }}/files/akamai_papi.py
      --operation get_active_version
      --edgerc {{ edgerc_path }}
      --section {{ akamai_edgerc_section }}
      --property-id {{ akamai_property_id }}
      --contract-id {{ akamai_contract_id }}
      --group-id {{ akamai_group_id }}
  register: active_version_result
  changed_when: false
  no_log: true

- name: "Set active version facts"
  ansible.builtin.set_fact:
    akamai_active_version_info: "{{ active_version_result.stdout | from_json }}"
    akamai_current_production_version: >-
      {{ (active_version_result.stdout | from_json).productionVersion | int }}
    akamai_current_staging_version: >-
      {{ (active_version_result.stdout | from_json).stagingVersion | int }}

- name: "Log currently active versions"
  ansible.builtin.debug:
    msg: >-
      Current versions -
      Production: {{ akamai_current_production_version }},
      Staging: {{ akamai_current_staging_version }}
```

File 13: roles/akamai_property/tasks/create_version.yml

yml

```

---
# Clone the currently active version for the target network into a new version
- name: "Determine base version to clone from"
  ansible.builtin.set_fact:
    akamai_base_version: >-
      {{ akamai_current_production_version
        if akamai_network == 'PRODUCTION'
        else akamai_current_staging_version }}

- name: "Create new property version (clone of v{{ akamai_base_version }})"
  ansible.builtin.command:
    cmd: >
      python3 {{ role_path }}/files/akamai_papi.py
      --operation create_version
      --edgerc {{ edgerc_path }}
      --section {{ akamai_edgerc_section }}
      --property-id {{ akamai_property_id }}
      --contract-id {{ akamai_contract_id }}
      --group-id {{ akamai_group_id }}
      --version {{ akamai_base_version }}
  register: create_version_result
  no_log: true

- name: "Set new version fact"
  ansible.builtin.set_fact:
    akamai_new_version: >-
      {{ (create_version_result.stdout | from_json).new_version | int }}

- name: "Log new version created"
  ansible.builtin.debug:
    msg: >-
      New version created: v{{ akamai_new_version }}
      (cloned from v{{ akamai_base_version }})

```

File 14: roles/akamai_property/tasks/diff.yml

yaml

```
---
# Fetch rule trees for both versions and generate a unified diff
- name: "Get rule tree for base version v{{ akamai_base_version }}"
  ansible.builtin.command:
    cmd: >
      python3 {{ role_path }}/files/akamai_papi.py
      --operation get_rules
      --edgerc {{ edgerc_path }}
      --section {{ akamai_edgerc_section }}
      --property-id {{ akamai_property_id }}
      --contract-id {{ akamai_contract_id }}
      --group-id {{ akamai_group_id }}
      --version {{ akamai_base_version }}
  register: base_rules_result
  changed_when: false
  no_log: true

- name: "Get rule tree for new version v{{ akamai_new_version }}"
  ansible.builtin.command:
    cmd: >
      python3 {{ role_path }}/files/akamai_papi.py
      --operation get_rules
      --edgerc {{ edgerc_path }}
      --section {{ akamai_edgerc_section }}
      --property-id {{ akamai_property_id }}
      --contract-id {{ akamai_contract_id }}
      --group-id {{ akamai_group_id }}
      --version {{ akamai_new_version }}
  register: new_rules_result
  changed_when: false
  no_log: true

- name: "Save base rules to temp file"
  ansible.builtin.copy:
    content: "{{ base_rules_result.stdout | from_json | to_nice_json }}"
    dest: "{{ artifacts_dir }}/v{{ akamai_base_version }}_rules.json"
    mode: "0644"

- name: "Save new rules to temp file"
  ansible.builtin.copy:
    content: "{{ new_rules_result.stdout | from_json | to_nice_json }}"
    dest: "{{ artifacts_dir }}/v{{ akamai_new_version }}_rules.json"
    mode: "0644"

- name: "Generate unified diff between versions"
  ansible.builtin.command:
```

```
cmd: >
    diff -u
    {{ artifacts_dir }}/v{{ akamai_base_version }}_rules.json
    {{ artifacts_dir }}/v{{ akamai_new_version }}_rules.json
register: diff_result
failed_when: diff_result.rc > 1    # diff returns 1 when differences found (no
changed_when: false

- name: "Save diff to artifacts"
  ansible.builtin.copy:
    content: "{{ diff_result.stdout }}"
    dest: >-
    {{ artifacts_dir }}/diff_v{{ akamai_base_version }}_to_v{{ akamai_new_ver
    mode: "0644"

- name: "Display diff for review"
  ansible.builtin.debug:
    msg: "{{ diff_result.stdout_lines }}"

- name: "Warn if no differences found"
  ansible.builtin.debug:
    msg: >-
    WARNING: No differences found between v{{ akamai_base_version }}
    and v{{ akamai_new_version }}. Are you sure you want to deploy?
  when: diff_result.stdout | length == 0
```

File 15: roles/akamai_property/tasks/activate.yml

yaml

```
---
# Activate the new version to STAGING or PRODUCTION
- name: "Activate v{{ akamai_new_version }} on {{ akamai_network }}"
  ansible.builtin.command:
    cmd: >
      python3 {{ role_path }}/files/akamai_papi.py
      --operation activate
      --edgerc {{ edgerc_path }}
      --section {{ akamai_edgerc_section }}
      --property-id {{ akamai_property_id }}
      --contract-id {{ akamai_contract_id }}
      --group-id {{ akamai_group_id }}
      --version {{ akamai_new_version }}
      --network {{ akamai_network }}
      --email {{ akamai_notify_email }}
      --note "SSPA automated deployment - CHG: {{ chg_number | default('N/A') }}
      {{ '--fast-fallback' if use_fast_fallback else '' }}"
  register: activate_result
  no_log: true

- name: "Set activation ID fact"
  ansible.builtin.set_fact:
    akamai_activation_id: >-
      {{ (activate_result.stdout | from_json).activation_id }}

- name: "Log activation started"
  ansible.builtin.debug:
    msg: >-
      Activation started. ID: {{ akamai_activation_id }}.
      Network: {{ akamai_network }}.
      Now polling every {{ activation_poll_interval }}s
      for up to {{ activation_poll_max_attempts * activation_poll_interval / 60
```

File 16: roles/akamai_property/tasks/poll_status.yml

yaml

```
---
# Poll until terminal state: ACTIVE, FAILED, or ABORTED
# Per Section 5.3 of the doc: MUST poll to terminal state,
# never treat the submission response as success
- name: "Poll activation status until terminal state"
  ansible.builtin.command:
    cmd: >
      python3 {{ role_path }}/files/akamai_papi.py
      --operation get_activation
      --edgerc {{ edgerc_path }}
      --section {{ akamai_edgerc_section }}
      --property-id {{ akamai_property_id }}
      --contract-id {{ akamai_contract_id }}
      --group-id {{ akamai_group_id }}
      --activation-id {{ akamai_activation_id }}
  register: poll_result
  until: >-
    (poll_result.stdout | from_json).status in ['ACTIVE', 'FAILED', 'ABORTED']
  retries: "{{ activation_poll_max_attempts }}"
  delay: "{{ activation_poll_interval }}"
  changed_when: false
  no_log: true

- name: "Set final activation status fact"
  ansible.builtin.set_fact:
    akamai_activation_status: "{{ poll_result.stdout | from_json }}"
    akamai_activation_final_status: "{{ (poll_result.stdout | from_json).status }}"

- name: "Log activation outcome"
  ansible.builtin.debug:
    msg: >-
      Activation {{ akamai_activation_id }} reached terminal state:
      {{ akamai_activation_final_status }}

- name: "Fail if activation did not succeed"
  ansible.builtin.fail:
    msg: >-
      ACTIVATION FAILED: v{{ akamai_new_version }} on {{ akamai_network }}
      reached status {{ akamai_activation_final_status }}.
      Activation ID: {{ akamai_activation_id }}.
      Check Akamai Control Center for details.
  when: akamai_activation_final_status != 'ACTIVE'
```

File 17: roles/akamai_property/tasks/rollback.yml

yaml

```

---
# Activate the previous stable version when current deployment fails
# Fast Fallback if within 60-min window, else normal activation
- name: "Determine rollback target version"
  ansible.builtin.set_fact:
    rollback_target_version: >-
      {{ akamai_current_production_version
        if akamai_network == 'PRODUCTION'
        else akamai_current_staging_version }}

- name: "Log rollback initiated"
  ansible.builtin.debug:
    msg: >-
      ROLLBACK INITIATED: Activating previous stable
      version v{{ rollback_target_version }} on {{ akamai_network }}

- name: "Activate previous stable version (rollback)"
  ansible.builtin.command:
    cmd: >
      python3 {{ role_path }}/files/akamai_papi.py
      --operation activate
      --edgerc {{ edgerc_path }}
      --section {{ akamai_edgerc_section }}
      --property-id {{ akamai_property_id }}
      --contract-id {{ akamai_contract_id }}
      --group-id {{ akamai_group_id }}
      --version {{ rollback_target_version }}
      --network {{ akamai_network }}
      --email {{ akamai_notify_email }}
      --note "ROLLBACK: Reverting to v{{ rollback_target_version }} - CHG: {{ c
      {{ '--fast-fallback' if use_fast_fallback else '' }}
  register: rollback_activate_result
  no_log: true

- name: "Set rollback activation ID"
  ansible.builtin.set_fact:
    rollback_activation_id: >-
      {{ (rollback_activate_result.stdout | from_json).activation_id }}

- name: "Poll rollback activation to completion"
  ansible.builtin.command:
    cmd: >
      python3 {{ role_path }}/files/akamai_papi.py
      --operation get_activation
      --edgerc {{ edgerc_path }}
      --section {{ akamai_edgerc_section }}

```

```
    --property-id {{ akamai_property_id }}
    --contract-id {{ akamai_contract_id }}
    --group-id {{ akamai_group_id }}
    --activation-id {{ rollback_activation_id }}
register: rollback_poll_result
until: >-
    (rollback_poll_result.stdout | from_json).status
    in ['ACTIVE', 'FAILED', 'ABORTED']
retries: "{{ activation_poll_max_attempts }}"
delay: "{{ activation_poll_interval }}"
changed_when: false
no_log: true

- name: "Set rollback final status"
  ansible.builtin.set_fact:
    rollback_final_status: >-
        {{ (rollback_poll_result.stdout | from_json).status }}

- name: "Log rollback outcome"
  ansible.builtin.debug:
    msg: "Rollback to v{{ rollback_target_version }}: {{ rollback_final_status
```

File 18: roles/snow_chg_validate/tasks/main.yml

(SSPA-4 + SSPA-8: Read-only CHG gate check)

yaml

```

---
# — ServiceNow CHG Validation —
# Purpose: Read-only gate check BEFORE any deployment.
# Blocks deployment unless:
#   1. CHG state == "implement"    (the CHG is actively being implemented)
#   2. CHG approval == "approved"  (a human has signed it off)
#   3. Current time is within scheduled start_date and end_date
# No writes to ServiceNow here – this role is purely a gate check.
# Per SSPA-4 acceptance criteria: "No ServiceNow records are modified"
#

- name: "Validate chg_number is provided"
  ansible.builtin.fail:
    msg: "chg_number is required for Production deployments. Pass it as an extra
    when: chg_number is not defined or chg_number | length == 0

- name: "Query ServiceNow change request {{ chg_number }}"
  servicenow.itsm.change_request_info:
    instance:
      host: "https://{{ snow_instance }}"
      username: "{{ vault_snow_username }}"
      password: "{{ vault_snow_password }}"
      number: "{{ chg_number }}"
    register: chg_query_result
    no_log: true    # don't log credentials
    retries: 3
    delay: 5
    until: chg_query_result is not failed

- name: "Fail if CHG not found"
  ansible.builtin.fail:
    msg: >-
      CHG VALIDATION FAILED: Change request {{ chg_number }}
      was not found in ServiceNow instance {{ snow_instance }}.
      Verify the change number is correct.
    when: chg_query_result.records | length == 0

- name: "Extract CHG fields"
  ansible.builtin.set_fact:
    chg_record: "{{ chg_query_result.records[0] }}"
    # Note: field names depend on your SNOW instance customization.
    # Confirm exact field names with your SNOW admin.
    # Common field names for PNC:
    chg_state: "{{ chg_query_result.records[0].state | lower }}"
    chg_approval: "{{ chg_query_result.records[0].approval | lower }}"
    chg_start: "{{ chg_query_result.records[0].start_date }}"

```

```

    chg_end: "{{ chg_query_result.records[0].end_date }}"

- name: "Log CHG details for audit"
  ansible.builtin.debug:
    msg: >-
      CHG {{ chg_number }} details -
      state: {{ chg_state }},
      approval: {{ chg_approval }},
      window: {{ chg_start }} to {{ chg_end }}

# — Validate state is "implement" —
# Per SSPA-8 acceptance criteria: "Only Implement status is accepted"
# Per your confirmation: need state=Implement AND approval=approved (both required)
#
- name: "Validate CHG state is 'implement'"
  ansible.builtin.assert:
    that:
      - chg_state == "implement"
    fail_msg: >-
      CHG GATE REJECTED: {{ chg_number }} state is '{{ chg_state }}'.
      Required: 'implement'. Deployment blocked.
    success_msg: "CHG state check PASSED: {{ chg_state }}"

# — Validate approval is "approved" —
- name: "Validate CHG approval is 'approved'"
  ansible.builtin.assert:
    that:
      - chg_approval == "approved"
    fail_msg: >-
      CHG GATE REJECTED: {{ chg_number }} approval is '{{ chg_approval }}'.
      Required: 'approved'. Deployment blocked.
    success_msg: "CHG approval check PASSED: {{ chg_approval }}"

# — Validate current time is within the scheduled window —
# Timestamps: confirm with SNOW admin whether fields are UTC or local TZ
# The comparison uses Ansible's ansible_date_time.iso8601 which is UTC
- name: "Get current UTC timestamp"
  ansible.builtin.set_fact:
    now_utc: "{{ ansible_date_time.iso8601 }}"

- name: "Validate deployment is within scheduled change window"
  ansible.builtin.assert:
    that:
      - now_utc >= chg_start
      - now_utc <= chg_end
    fail_msg: >-
      CHG GATE REJECTED: Current time {{ now_utc }} is outside the

```

```
        scheduled change window for {{ chg_number }}.
        Window: {{ chg_start }} to {{ chg_end }}.
        Deployment blocked.
    success_msg: >-
        CHG window check PASSED: {{ now_utc }} is within
        {{ chg_start }} → {{ chg_end }}

- name: "Log CHG validation success"
  ansible.builtin.debug:
    msg: >-
        ✓ CHG VALIDATION PASSED: {{ chg_number }}
        is Approved, in Implement state,
        and within scheduled window. Deployment authorized.
```

File 19: roles/snow_ctask_update/tasks/main.yml

(SSPA-14: Write deployment results back to ctask)

yaml

```

---
# — ServiceNow ctask Note Update —
# Purpose: Write deployment execution results to the change_task (ctask)
# record AFTER deployment completes (success or failure).
# Uses work_notes field (internal, append-only journal – does NOT overwrite).
# Per SSPA-14 mandatory fields:
#   - Version Number
#   - Triggered By (email)
#   - Triggered On (UTC timestamp)
#   - Status (PASS/FAIL)
# Optional fields (if applicable):
#   - Rollback Information
#   - Error Logs / Deployment Details
#

- name: "Validate ctask_number is provided"
  ansible.builtin.fail:
    msg: "ctask_number is required for ctask note updates."
  when: ctask_number is not defined or ctask_number | length == 0

# Build the note content (mandatory fields always present)
- name: "Build deployment note content"
  ansible.builtin.set_fact:
    deployment_note: |
      === AUTOMATED DEPLOYMENT REPORT ===
      Timestamp (UTC):      {{ ansible_date_time.iso8601 }}
      Triggered By:        {{ akamai_notify_email }}
      CHG Number:          {{ chg_number | default('N/A') }}
      Environment:         {{ akamai_network }}
      Property ID:         {{ akamai_property_id }}
      Version Deployed:    v{{ akamai_new_version | default('N/A') }}
      Previous Version:    v{{ akamai_base_version | default('N/A') }}
      Deployment Status:   {{ deployment_status | default('UNKNOWN') }}
      Activation ID:       {{ akamai_activation_id | default('N/A') }}
      Activation Result:   {{ akamai_activation_final_status | default('N/A') }}
      {% if deployment_status == 'FAIL' %}
      === FAILURE DETAILS ===
      Error:                {{ deployment_error | default('See Akamai Control Cen
      {% endif %}
      {% if rollback_performed is defined and rollback_performed %}
      === ROLLBACK INFORMATION ===
      Rollback Version:    v{{ rollback_target_version | default('N/A') }}
      Rollback Status:     {{ rollback_final_status | default('N/A') }}
      Rollback Timestamp:  {{ ansible_date_time.iso8601 }}
      {% endif %}
      === END REPORT ===

```

```

- name: "Update ctask work_notes with deployment result"
  servicenow.servicenow.snow_record:
    instance: "{{ snow_instance }}"
    username: "{{ vault_snow_username }}"
    password: "{{ vault_snow_password }}"
    table: "change_task"
    number: "{{ ctask_number }}"
    state: present
    data:
      # work_notes is an append-only journal field in ServiceNow
      # Each write adds a new timestamped entry – does NOT overwrite
      work_notes: "{{ deployment_note }}"
  register: ctask_update_result
  no_log: true
  retries: 3
  delay: 5
  until: ctask_update_result is not failed

- name: "Log ctask update success"
  ansible.builtin.debug:
    msg: >-
      ctask {{ ctask_number }} work_notes updated successfully
      with {{ deployment_status }} deployment result.

- name: "Warn if ctask update failed after retries"
  ansible.builtin.debug:
    msg: >-
      WARNING: Failed to update ctask {{ ctask_number }} after 3 retries.
      Manual update required. Deployment result was: {{ deployment_status }}
  when: ctask_update_result is failed

```

File 20: playbooks/deploy_akamai_config.yml

(The main orchestrating playbook — ties everything together)

```

yaml

```

```
---
# -----
# SSPA-1: Akamai Property Manager Config Push
#
# Usage:
#   Non-Production:
#     ansible-playbook -i inventories/staging playbooks/deploy_akamai_config.yml
#
#   Production (requires CHG number):
#     ansible-playbook -i inventories/production playbooks/deploy_akamai_config
#       -e "chg_number=CHG0012345 ctask_number=CTASK0012345"
#
# Required extra-vars for Production:
#   chg_number:    ServiceNow Change Request number (e.g. CHG0012345)
#   ctask_number:  ServiceNow Change Task number      (e.g. CTASK0012345)
# -----
- name: "SSPA-1: Akamai Config Deployment"
  hosts: localhost
  gather_facts: true      # needed for ansible_date_time.iso8601 timezone check
  connection: local

  vars:
    deployment_status: "UNKNOWN"
    deployment_error: ""
    rollback_performed: false

  pre_tasks:
    # ----- Step 1: Determine if this is Production or Non-Production -----
    - name: "Log deployment start"
      ansible.builtin.debug:
        msg: >-
          Starting SSPA-1 Akamai Config Push.
          Target network: {{ akamai_network }}.
          Environment: {{ environment_name }}

    # ----- Step 2: PRODUCTION GATE – ServiceNow CHG validation -----
    # This only runs when deploying to Production
    # Per the Confluence workflow doc:
    #   - Change Number must be provided
    #   - Change must be in Approved state
    #   - Deployment must occur within Scheduled Change Window
    - name: "Production gate: Validate ServiceNow change request"
      ansible.builtin.include_role:
        name: snow_chg_validate
      when: akamai_network == "PRODUCTION"
```

```
- name: "Non-Production: Log JIRA reference if provided"
  ansible.builtin.debug:
    msg: >-
      Non-Production deployment.
      JIRA reference: {{ jira_ticket | default('Not provided (optional)') }}
  when: akamai_network != "PRODUCTION"
```

tasks:

```
# — Step 3 onward: Run the Akamai deployment role —————
# Wrapped in block/rescue for automatic rollback on failure
- name: "Akamai deployment block"
  block:
    # Preflight → Create version → Diff → Activate → Poll
    - name: "Execute Akamai property push"
      ansible.builtin.include_role:
        name: akamai_property

    # If we get here, deployment succeeded
    - name: "Set deployment status to PASS"
      ansible.builtin.set_fact:
        deployment_status: "PASS"

    - name: "Log deployment success"
      ansible.builtin.debug:
        msg: >-
          ✓ DEPLOYMENT SUCCEEDED: v{{ akamai_new_version }}
            is now ACTIVE on {{ akamai_network }}.
            Property: {{ akamai_property_id }}
            CHG: {{ chg_number | default('N/A') }}

  rescue:
    # — Failure path —————
    - name: "Set deployment status to FAIL"
      ansible.builtin.set_fact:
        deployment_status: "FAIL"
        deployment_error: "{{ ansible_failed_result.msg | default('Unknown') }}"

    - name: "Log deployment failure"
      ansible.builtin.debug:
        msg: >-
          ✗ DEPLOYMENT FAILED: {{ deployment_error }}
            Checking rollback configuration...

    # — Automatic rollback —————
    - name: "Execute rollback (if enabled and version was created)"
      ansible.builtin.include_role:
        name: akamai_property
```

```

    tasks_from: rollback.yml
  when:
    - rollback_on_failure | bool
    - akamai_new_version is defined      # only if we got past create_ver

- name: "Mark rollback as performed"
  ansible.builtin.set_fact:
    rollback_performed: true
  when:
    - rollback_on_failure | bool
    - akamai_new_version is defined

```

post_tasks:

```

# — Step 7 (Confluence doc): Update ctask with results —————
# Always runs regardless of deployment success or failure
# Only runs on Production deployments (ctask_number required)
- name: "Update ServiceNow ctask with deployment results"
  ansible.builtin.include_role:
    name: snow_ctask_update
  when:
    - akamai_network == "PRODUCTION"
    - ctask_number is defined
    - ctask_number | length > 0

```

```

# — Cleanup: Delete temporary .edgerc file —————
# Per Section 8.7: .edgerc is never stored permanently
- name: "Delete temporary .edgerc (always runs)"
  ansible.builtin.file:
    path: "{{ edgerc_path }}"
    state: absent
  no_log: true

```

```

# — Final summary —————
- name: "Deployment summary"
  ansible.builtin.debug:
    msg:
      - "—————"
      - "SSPA-1 Deployment Summary"
      - "—————"
      - "Status:          {{ deployment_status }}"
      - "Network:         {{ akamai_network }}"
      - "Property:        {{ akamai_property_id }}"
      - "Previous version: v{{ akamai_base_version | default('unknown') }}"
      - "New version:      v{{ akamai_new_version | default('N/A - create_ver') }}"
      - "CHG:             {{ chg_number | default('N/A') }}"
      - "ctask:           {{ ctask_number | default('N/A') }}"
      - "Rollback:        {{ 'YES - v' + (rollback_target_version | string) }}"

```

```
- "_____"  
  
# Propagate failure so AAP marks the job as failed  
- name: "Fail the play if deployment was unsuccessful"  
  ansible.builtin.fail:  
    msg: "Deployment failed. See summary above."  
    when: deployment status == "FAIL"
```

How to Run This

Install dependencies first (once):

```
bash  
  
# Python deps  
pip install edgegrid-python requests pysnow --break-system-packages  
  
# Ansible collections  
ansible-galaxy collection install -r collections/requirements.yml
```

Encrypt your secrets:

```
bash  
  
# Create vault password file  
echo "your_vault_password_here" > .vault_pass  
chmod 600 .vault_pass  
  
# Encrypt your vault.yml files  
ansible-vault encrypt inventories/staging/group_vars/all/vault.yml  
ansible-vault encrypt inventories/production/group_vars/all/vault.yml
```

Run for Non-Production (no CHG needed):

```
bash  
  
ansible-playbook -i inventories/staging playbooks/deploy_akamai_config.yml
```

Run for Production (CHG mandatory):

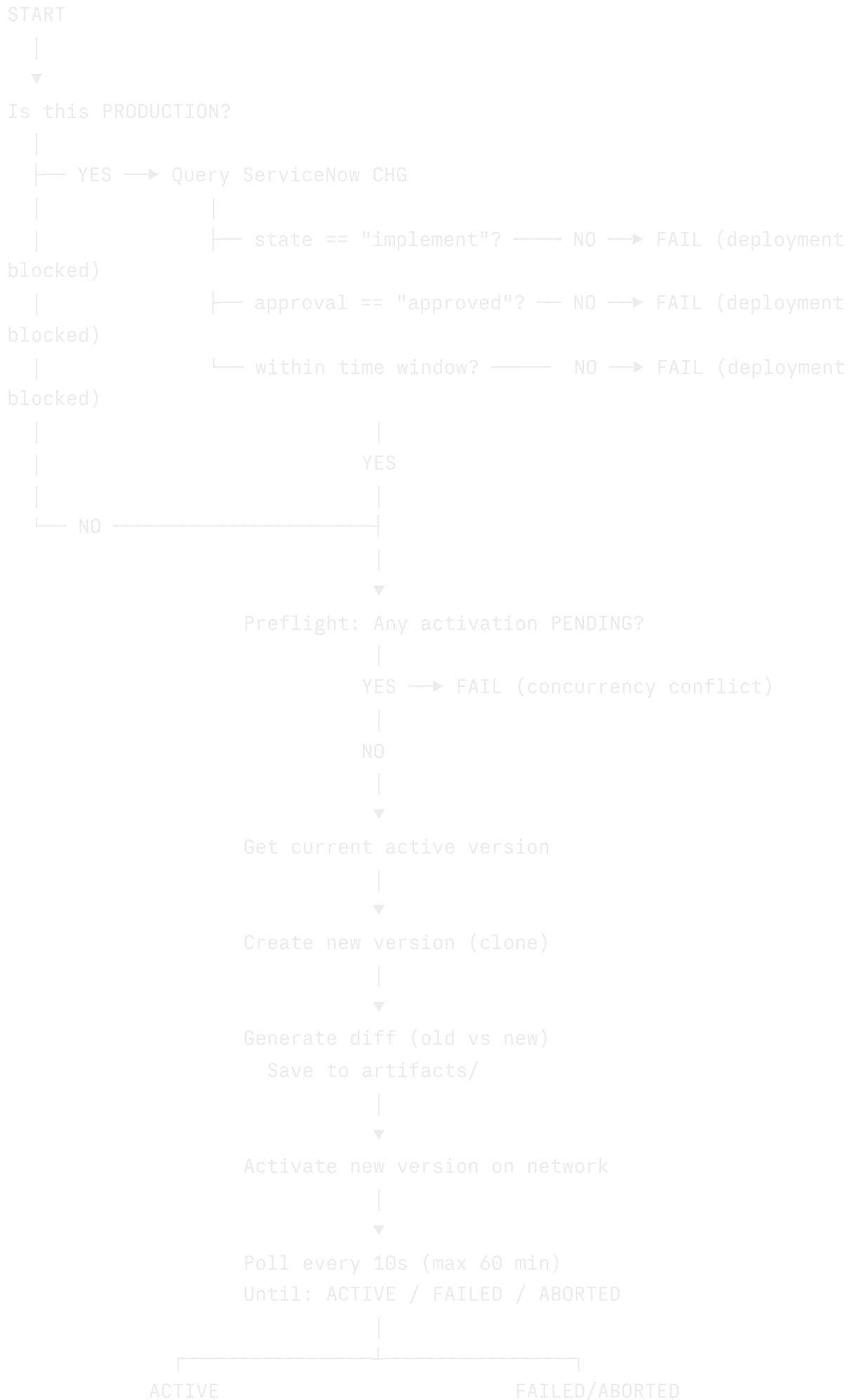
```
bash  
  
ansible-playbook -i inventories/production playbooks/deploy_akamai_config.yml \  
-e "chg_number=CHG0012345 ctask_number=CTASK0098765"
```

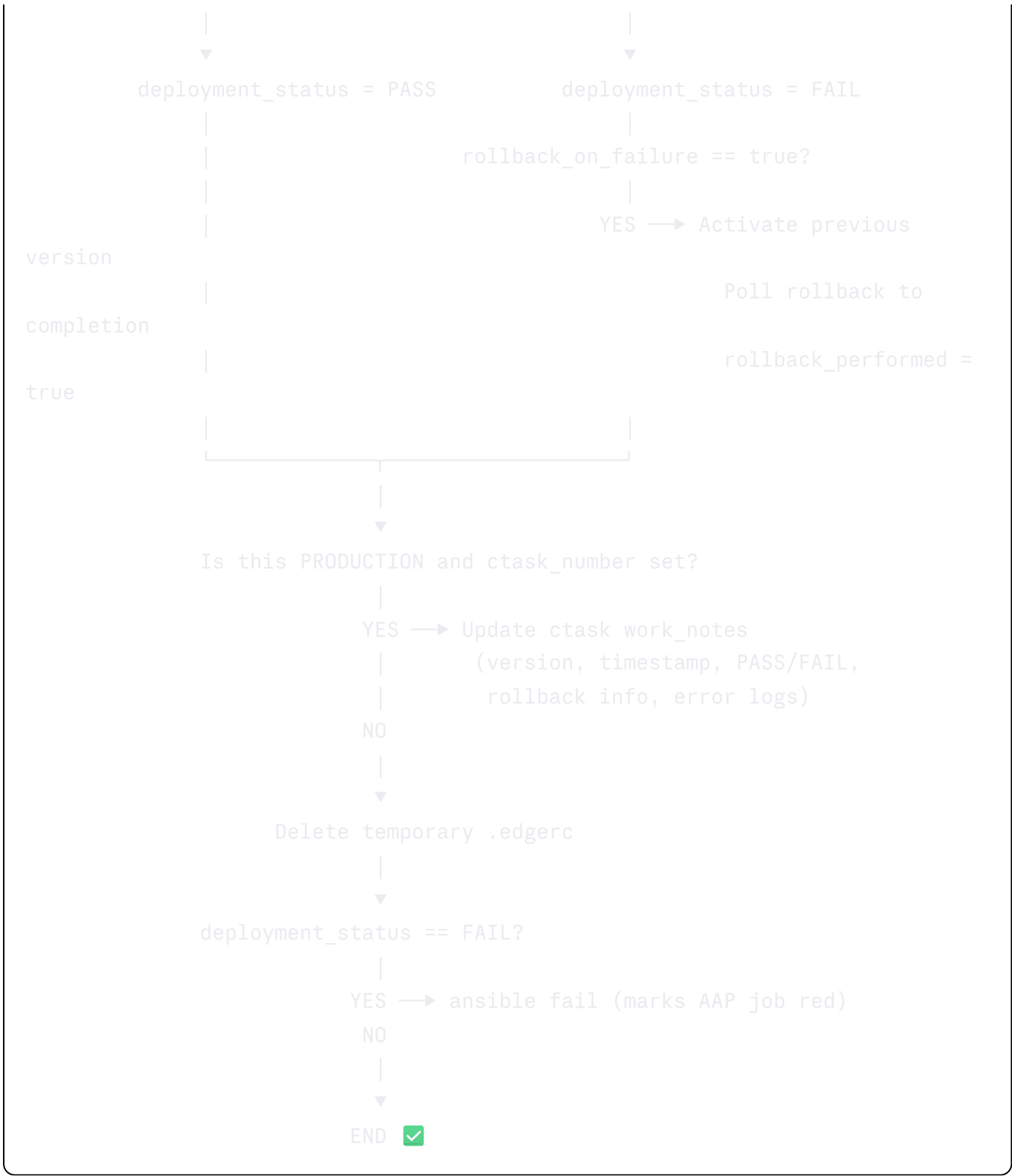
Syntax check only:

```
bash
```

```
ansible-playbook --syntax-check \  
-i inventories/staging \  
playbooks/deploy_akamai_config.yml
```

The Complete Flow Visualized





Which SSPA Tickets Map to Which Files

Ticket	What it requires	Files that implement it
SSPA-4	Query SNOW CHG, get state + approval + window	<code>roles/snow_chg_validate/tasks/main.yml</code>
SSPA-8	Validate state==implement AND approval==approved, reject with error	<code>roles/snow_chg_validate/tasks/main.yml</code> (assert tasks)
SSPA-14	Update ctask work_notes with deployment results post-execution	<code>roles/snow_ctask_update/tasks/main.yml</code>
SSPA-1 (epic)	The full pipeline end-to-end	<code>playbooks/deploy_akamai_config.yml</code> + all Akamai role tasks

Things to Confirm with Onshore Before Running

1. **Exact SNOW field names** — `state`, `approval`, `start_date`, `end_date` may be customised at PNC. Ask your SNOW admin.
2. **State string values** — Does SNOW return `"implement"` (lowercase) or `"Implement"` or an integer code? Print a raw API response against a test CHG to check.
3. **Timezone of SNOW date fields** — UTC or local? Must match for the window check to work.
4. **ctask table name** — could be `change_task` or a custom table. Check with SNOW admin.
5. `servicenow.itsm` in your EE — confirm the collection is installed in your AAP Execution Environment, or add it to a custom EE build.
6. `contractId` and `groupId` for BOP mnemonic — get the exact values from the Akamai TAM.
7. **API client credentials** — READ-WRITE grant needed, separate clients for staging and production.
8. **Egress IP allowlisting** — AAP automation host IP must be whitelisted to reach `akab-xxxx.luna.akamaiapis.net`.